

1. React DOM

¿Qué es React DOM?

React DOM (react-dom) es el paquete específico que actúa como pegamento o intermediario entre la biblioteca principal de React y el DOM (*Document Object Model*) real del navegador web. Mientras que el núcleo de React se encarga de definir componentes, manejar el estado y construir el árbol del Virtual DOM de manera agnóstica a la plataforma, React DOM contiene los métodos concretos para traducir, proyectar e inyectar esos componentes en un entorno web de navegador.

Características principales

- **Punto de montaje del Virtual DOM:** Toma la estructura en memoria generada por React y la acopla al nodo raíz del HTML del documento.
- **Algoritmo de reconciliación eficiente:** Trabaja en conjunto con React para actualizar en el DOM real únicamente las partes exactas que cambiaron, evitando repintados innecesarios de la página.
- **API moderna basada en raíces (createRoot):** Desde React 18 en adelante, utiliza ReactDOM.createRoot() para habilitar las características de concurrencia de React, reemplazando el método legado ReactDOM.render().
- **Capacidades para renderizado en servidor (SSR):** A través de subpaquetes como react-dom/server, permite convertir componentes de React a cadenas HTML en el servidor para optimizar el rendimiento y el SEO.
- **Gestión de Portales (createPortal):** Proporciona la capacidad de renderizar un componente secundario en un nodo DOM diferente y fuera de la jerarquía de su componente padre (muy utilizado para ventanas modales y tooltips).

¿Para qué sirve?

Sirve para inicializar la aplicación web montando el componente principal dentro del contenedor HTML del navegador (<div id="root"></div>), y para sincronizar en tiempo real los cambios del Virtual DOM con los elementos visuales que ve el usuario.

```
import React from 'react';
import ReactDOM from 'react-dom/client';

function App() {
  return (
    <div style={{ fontFamily: 'Arial, sans-serif', padding: '20px' }}>
      <h1>Sistema de Gestión Académica</h1>
      <p>Aplicación montada con éxito en el navegador mediante
ReactDOM.</p>
    </div>
  );
}
```

```
const contenedor = document.getElementById('root');
const raiz = ReactDOM.createRoot(contenedor);
raiz.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

- **document.getElementById('root')**: Utiliza la API nativa de JavaScript para buscar el elemento contenedor base definido en el HTML (<div id="root"></div>).
- **ReactDOM.createRoot(contenedor)**: Inicializa el punto de entrada de la aplicación en el DOM del navegador y prepara el motor de concurrencia de React.
- **raiz.render(...)**: Indica a React DOM que tome el componente <App/>, transforme sus elementos JSX en nodos reales del navegador y los inserte dentro del contenedor #root.

2. React Router

¿Qué es React Router?

React Router (comúnmente a través del paquete react-router-dom) es la biblioteca estándar de facto para la gestión de enrutamiento y navegación en aplicaciones React de una sola página (*Single Page Applications* o SPAs). Permite cambiar de vistas y componentes dinámicamente según la URL escrita en la barra de direcciones del navegador, sin provocar una recarga completa de la página.

Características principales

- **Navegación del lado del cliente (Client-Side Routing)**: Modifica la URL y renderiza nuevas pantallas interceptando los enlaces mediante la API de historial de HTML5 (window.history), manteniendo la fluidez de una aplicación de escritorio.
- **Enrutamiento declarativo basado en componentes**: Las rutas se configuran como componentes nativos de React (<BrowserRouter>, <Routes>, <Route>).
- **Componente <Link>**: Sustituye a la etiqueta tradicional de HTML, previniendo la recarga total del navegador al hacer clic y permitiendo transiciones instantáneas.
- **Parámetros dinámicos en URL**: Permite definir segmentos dinámicos de ruta (por ejemplo, /productos/:id) y leerlos directamente en los componentes mediante el hook useParams.
- **Rutas anidadas y compartición de diseño**: Permite anidar rutas hijas y representarlas dentro de componentes envolventes mediante la directiva <Outlet>.

¿Para qué sirve?

Sirve para estructurar la navegación de una aplicación web moderna con múltiples vistas (Inicio, Productos, Perfil, Contacto), sincronizando la URL del navegador con los componentes que deben mostrarse en pantalla y habilitando el uso normal de los botones "Atrás" y "Adelante" del navegador.

```

import React from 'react';
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

function Inicio() {
  return <h2>Página de Inicio - Catálogo General</h2>;
}

function Contacto() {
  return <h2>Página de Contacto - Soporte Técnico</h2>;
}

function NoEncontrado() {
  return <h2>Error 404: Página no encontrada</h2>;
}

function App() {
  return (
    <BrowserRouter>
      <div>
        <nav style={{ padding: '10px', borderBottom: '1px solid #ccc' }}>
          <Link to="/" style={{ marginRight: '15px' }}>Inicio</Link>
          <Link to="/contacto">Contacto</Link>
        </nav>

        <div style={{ padding: '20px' }}>
          <Routes>
            <Route path="/" element={<Inicio />} />
            <Route path="/contacto" element={<Contacto />} />
            <Route path="*" element={<NoEncontrado />} />
          </Routes>
        </div>
      </div>
    </BrowserRouter>
  );
}

export default App;

```

Explicación del ejemplo:

1. `<BrowserRouter>`: Envuelve la aplicación y provee el contexto para conectar React con el historial del navegador.
2. `<Link to="...">`: Genera hipervínculos que actualizan la URL en el navegador evitando que la página se reinicie por completo.
3. `<Routes>` y `<Route>`: Definen la tabla de ruteo. La etiqueta `<Routes>` inspecciona la URL actual y busca el `<Route>` cuya propiedad `path` coincida; cuando hay coincidencia, monta de inmediato en la pantalla el componente indicado en la propiedad `element`.
4. `path="*"`: Actúa como ruta comodín (*fallback*) para capturar cualquier dirección no contemplada y renderizar una vista personalizada de error 404.